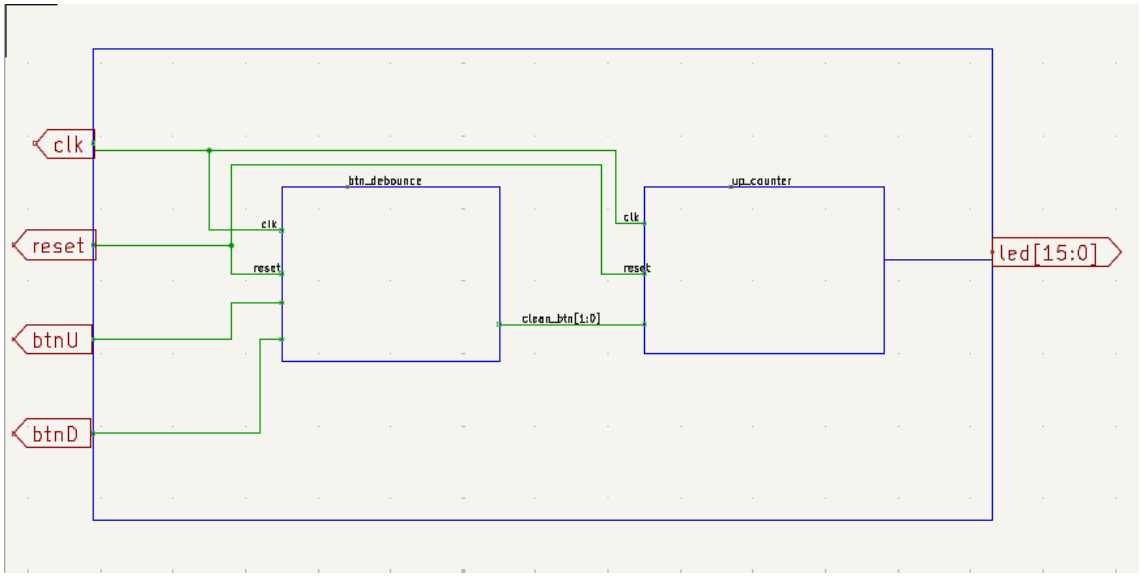

clk
reset
btnU
btnD
btn_debounce
clk
reset
clean_btn[1:0]
up_counter
clk
reset
led[15:0]

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2026/10/16 14:38:00
// Design Name:
// Module Name: top_up_counter
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module top_up_counter(
    input wire clk,
    input wire reset,
    input wire btnU,
    input wire btnD,
    output wire [15:0] led
);
    wire [1:0] w_clean_btn;
    button_debounce u_debounce (
        .clk(clk),
        .reset(reset),
        .btnU(btnU),
        .btnD(btnD),
        .clean_btn(w_clean_btn)
    );
    up_counter u_counter (
        .clk(clk),
```

```
        .reset(reset),
        .btnU(w_clean_btn[0]),
        .btnD(w_clean_btn[1]),
        .led(led)
    );
endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2026/10/15 14:19:12
// Design Name:
// Module Name: button_debounce
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module button_debounce #(
    parameter DIV =1000000
)(
    input wire clk,
    input wire reset,
    input wire btnU,
    input wire btnD,
    output reg [1:0] clean_btn
);
    reg [$clog2(DIV)-1:0] r_count_u;
    reg [$clog2(DIV)-1:0] r_count_d;
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            r_count_u    <=0;
            clean_btn[0] <=1'b0;
        end
        else if (r_count_u ==DIV -1) begin
```

```
            r_count_u    <=0;
            clean_btn[0] <=btnU;
        end
        else begin
            r_count_u    <=r_count_u +1;
        end
    end
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            r_count_d    <=0;
            clean_btn[1] <=1'b0;
        end
        else if (r_count_d ==DIV -1) begin
            r_count_d    <=0;
            clean_btn[1] <=btnD;
        end
        else begin
            r_count_d    <=r_count_d +1;
        end
    end
endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 2026/10/16 14:38:00
// Design Name:
// Module Name: up_counter
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module up_counter #(
    parameter DIV =1000000
)(
    input wire clk,
    input wire reset,
    input wire btnU,          // 상태 토글용 RUN <-> STOP
    input wire btnD,          // STOP 상태에서 리셋용
    output reg [15:0] led
);
    parameter RUN  =1'b0;
    parameter STOP =1'b1;
    reg state =RUN;

    reg r_btnU =1'b0;
    wire w_btnU;
    reg [$clog2(DIV)-1:0] r_div_count =0;
    wire w_tick_10ms = (r_div_count ==DIV -1);
```

```
always @(posedge clk or posedge reset) begin
    if (reset) begin
        r_div_count <=0;
    end
    else if (w_tick_10ms) begin
        r_div_count <=0;
    end
    else begin
        r_div_count <=r_div_count +1;
    end
end
always @(posedge clk or posedge reset) begin
    if (reset) begin
        r_btnU <=1'b0;
    end else begin
        r_btnU <=btnU;
    end
end
assign w_btnU = (btnU &&!r_btnU); // 0에서 1로 변하는 순간 펄스 발생???
always @(posedge clk or posedge reset) begin
    if (reset) begin
        state <=RUN;
        led   <=16'b0;
    end
    else begin
        if (w_btnU) begin
            state <=~state;
        end
        case (state)
            RUN: begin
                if (w_tick_10ms) begin
                    led <=led +1;
                end
            end
            STOP: begin
                if (btnD) begin
                    led <=16'b0;
                end
            end
```

```
            endcase
        end
    end
endmodule
```